QL 文法

赵银亮

28/4/26

$P \longrightarrow B$ //语义约束为：$P \Rightarrow^{+} \{(TV;)^{*}(HB;)^{*}\check{S}\}$

$B \longrightarrow \{\check{D}\check{S}\}$

$T \longrightarrow \text{int} \mid \text{void}$

$D \longrightarrow TV \mid HB$ //语义约束为：B 中不能有函数声明

$H \longrightarrow T\boldsymbol{d}(\check{A})$

$S \longrightarrow ; \mid E; \mid \text{if}(C)S \mid \text{if}(C)S \text{ else } S \mid \text{while}(C)S \mid \text{return } E; \mid B$ // B 退化为$\{\check{S}\}$

$V \longrightarrow \boldsymbol{d} \mid V[E]$

$E \longrightarrow \boldsymbol{i} \mid V \mid \boldsymbol{d}(\check{R}) \mid E\boldsymbol{o}E \mid (E)$

$\check{D} \longrightarrow \varepsilon \mid \check{D}D;$

$\check{A} \longrightarrow \varepsilon \mid \check{A}A;$

$\check{S} \longrightarrow S \mid \check{S};S$

$\check{R} \longrightarrow \varepsilon \mid \check{R}E,$

$A \longrightarrow T\boldsymbol{d} \mid T\boldsymbol{d}[]$ //没有函数作形参

$C \longrightarrow E \mid E\boldsymbol{r}E \mid !C \mid C\&\&C \mid C||C \mid (C)$

//其中：$\boldsymbol{i}$ =[0-9]+；$\boldsymbol{d}$ =[a-zA-Z]+ ；$\boldsymbol{o}$={+,−,×,/,=,+=}。

测试程序 qsort

```
{
int a[10];

void swap(int arr[]; int i; int j;) {
    int temp;
    temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp; }

int partition(int arr[]; int low; int high;) {
     // 基准选最后一个元素
    int pivot;
    int i;
    int j;
    pivot = arr[high];
    i = low - 1;
    j = low - 1;
    while ( (j = j + 1) < high ) {
        if (arr[j] < pivot) {
            i = i + 1; swap(arr[], i, j,); } }
    swap(arr[], i+1, high,);
    return i + 1; }

void qsort(int arr[]; int low; int high;){
    int p;
    if (low < high) {
    p = partition(arr, low, high,);
    qsort(arr, low, p – 1,);
    qsort(arr, p + 1, high,); } }

qsort(a[], 0, 9,);
}
```

qsort 的 QTAC 代码

```
i80 a;
define swap(arr, i, j){
LABEL swap;
t0 = arr + i;
temp = M[t0];
t1 = arr + i
t2 = arr + j;
t3 = M[t2];
M[t1] = t3;
t4 = arr + j;
M[t4] = temp;
RETURN 0;}

define partition(arr, low, high){
LABEL partition;
t5 = arr + high;
pivot = M[t5];
t6 = low - 1;
i = t6;
t7 = low - 1;
j = t7;
LABEL l0;
t8 = j + 1;
j = t8;
IF t8 < high THEN l1 ELSE l2;
LABEL l1;
t9 = arr + j
t10 = M[t9];
IF t10 < pivot THEN l3 ELSE l4;
LABEL l3;
t11 = i + 1;
i = t11;
PAR arr; PAR i; PAR j;
t12 = CALL swap, 3;
GOTO l0;
LABEL l4;
GOTO l0;
LABEL l2;
t13 = arr + j;
t14 = i + 1;
PAR t13; PAR t14; PAR high;
```

```
t15 = CALL swap, 3;
t16 = i + 1;
RETURN t16;}

define qsort(arr, low, high){
LABEL qsort;
IF low < high THEN l5 ELSE l6;
LABEL l5;
PAR arr; PAR low; PAR high;
p = CALL partition, 3;
t17 = p - 1;
PAR arr; PAR low; PAR t17;
t18 = CALL qsort, 3;
t19 = p + 1;
PAR arr; PAR t19; PAR high;
t20 = CALL qsort, 3;
LABEL l6;
RETURN 0;
}
PAR a; PAR 0; PAR 9;
t21 = CALL qsort, 3;
```

指令模板 Q2ARM

赵银亮
西安交通大学
1/4/26; 28/4/26

$\mathcal{L}$（QTAC 指令模板）	ARM64 指令生成模式
<T_RETURN>⟶RETURN a;	ADR/MOV X0,X_a; b ⟨d@exit⟩; (a is l)ADR (otherwise)MOV
<T_PAR_CALL>⟶(PARa;){0,8}q=CALL d,k;	ADR/MOV X0,X_{a1}; …; ADR/MOV Xk,X_{ak}; BL d; MOV X_q,X0; (a is l)ADR (otherwise)MOV
<T_GOTO>⟶GOTO e;	(e is l) B l; (e is q) BR X_q;
<T_LABEL>⟶LABEL l;	l:
<T_IF_LABEL>⟶IF q ⟨rop⟩ a THEN l_1 ELSE l_2; LABEL l_3;	(l_3=l_2) CMP X_q, X_a; ⟨Jrop⟩ l_1; l_2: (l_3=l_1) CMP X_q, X_a; ⟨Jrop'⟩ l_2; l_1:
<T_IF>⟶IF q ⟨rop⟩ a THEN l_1 ELSE l_2;	CMP X_q, X_a; ⟨Jrop⟩ l_1; B l_2;
<T_LDR0>⟶q_1 = M[q_2];	LDR X_{q1}, [X_{q2}];
<T_LDR2>⟶q_1 = q_2 + b; q_3 = M[q_1];	(q_1=q_2 ∧b is k) LDR X_{q3}, [X_{q1},#k]!; (q_1^{last}≠q_2) LDR X_{q3}, [X_{q2},X_b];
<T_STR0>⟶M[q] = a;	(a is l) ADR ⟨Xcrs⟩,a; STR X_q,[⟨Xcrs⟩]; (otherwise) STR X_a, [X_q];
<T_STR2>⟶q_1 = q_2 + b; M[q_1] = a;	(q_1=q_2 ∧b is k) STR X_a, [X_{q1},#k]!; (q_1^{last}≠q_2 ∧a is l) ADR ⟨Xcrs⟩,a; STR ⟨Xcrs⟩, [X_{q1},X_b]; (q_1^{last}≠q_2) STR X_a, [X_{q1}, X_a];

<T_AOP>⟶q_1 = q_2 ⟨aop⟩ a;	⟨Xaop⟩ X_{q1}, X_{q2}, X_a;
<T_MOVE>⟶q = a;	(a isnot l) MOV X_q, X_a; (a is l) ADR X_q, X_a;
⟨QTAC-program⟩⟶ik_1 d_1; ...; ik_n d_n ⟨QTAC-func-list⟩ ⟨QTAC-ins-list⟩	.section .data .align 3 d_1:.skip k_1 ... d_n:.skip k_n .section .text .global _start ⟨QTAC-func-list-to-ARM⟩ _start: ⟨QTAC-ins-list-to-ARM⟩ MOV X0, #0 MOV X8, #93 SVC #0
⟨QTAC-func⟩⟶define d(a_1, ..., a_n){ LABEL d; ⟨QTAC-ins-list⟩}	d: MOV X_{a1},X0; ...; MOV X_{ak},Xk; stp x29, x30, [sp, #-16]!; mov x29, sp; sub sp, sp, #⟨d@width⟩; ⟨QTAC-ins-list-to-ARM⟩ ⟨d@exit⟩: add sp, sp, #⟨d@width⟩; ldp x29, x30, [sp], #16; ret;

⟨Jrop⟩：B.EQ、B.NE、B.GE、B.GT、B.LE、B.LT

⟨Jrop'⟩：B.NE、B.EQ、B.LT、B.LE、B.GT、B.GE

⟨Xaop⟩：ADD、SUB、MUL、SDIV　　⟨Kaop⟩：ADD、SUB

⟨Xcrs⟩虚拟寄存器 X_t

X_a(a is q) = X_q

X_a(a is k) = #k

X_a(a is l) = l

t 是临时变量，即⟨temp⟩；t^{last} 与 t 没有区别（让 QTAC 代码生成程序来保证）；

如果 x 是虚拟寄存器，那么 X_x 就是 x，否则 X_x 是虚拟寄存器。正体 X 开头的指物理寄存器。

⟨d@width⟩为函数 d 的类型宽度，即局部区的大小。

⟨QTAC-ins-list-to-ARM⟩是对⟨QTAC-ins-list⟩按照本模板进行转换得到的 ARM 指令段。

⟨QTAC-func-list-to-ARM⟩是对⟨QTAC-func-list⟩按照本模板进行转换得到的 ARM 指令段。

⟨d@exit⟩是当前函数 d 的尾声。

阶段	汇编代码模板	解释
序言	stp x29, x30, [sp, #-16]!	保存 FP 和 LR：将旧的帧指针和返回地址压入栈中，同时调整栈指针（SP）。！ 表示写回 SP。
(Prologue)	mov x29, sp	建立新帧：将当前的 SP 赋值给 FP，确立当前栈帧的基准。
	sub sp, sp, #N	分配局部变量：如果需要局部变量，继续减小 SP，分配 N 字节空间（需保持 16 字节对齐）。
尾声	add sp, sp, #N	释放局部变量：恢复 SP 到 FP 的位置（如果有局部变量）。
(Epilogue)	ldp x29, x30, [sp], #16	恢复 FP 和 LR：从栈中弹出旧的 FP 和 LR，同时恢复 SP（#16 表示后索引加）。
	ret	返回：跳转到 LR 指向的地址。

参数寄存器：X0 – X7
返回值：X0
Caller-saved：X9 – X15
Callee-saved：X19 – X29（需在 prologue 保存）
临时地址寄存器：X16 – X17（可用于 ADR）

中间语言 QTAC

赵银亮

1/4/26; 28/4/26

⟨QTAC-program⟩⟶⟨QTAC-global-list⟩⟨QTAC-func-list⟩ ⟨QTAC-ins-list⟩

⟨QTAC-global-list⟩⟶ε | ⟨QTAC-global⟩⟨QTAC-global-list⟩

⟨QTAC-global⟩⟶ik d;

⟨QTAC-func-list⟩⟶ε | ⟨QTAC-func⟩⟨QTAC-func-list⟩

⟨QTAC-func⟩⟶define d(a_1, ..., a_n){LABEL d; ⟨QTAC-ins-list⟩} \\0⩽n⩽8

⟨QTAC-ins-list⟩⟶⟨QTAC-ins⟩;|⟨QTAC-ins-list⟩⟨QTAC-ins⟩;

⟨QTAC-ins⟩⟶LABEL l | GOTO e | IF q ⟨rop⟩ a THEN l ELSE l

q⟶d | ⟨temp⟩

a⟶q | k | l

b⟶q | k

e⟶q | l

⟨QTAC-ins⟩⟶PAR a | q = CALL d, k | RETURN a

⟨QTAC-ins⟩⟶q = q ⟨aop⟩ a | q = a

⟨QTAC-ins⟩⟶q = M[q] | M[q] = a

⟨TAC-ins⟩⟶ NOP | q = CONVERT q, t

k 是立即数，k∈[0, 4095]；t 为 INT 或 FLOAT；l 为地址，形式为 li；⟨temp⟩为临时变量名，形式为 ti。⟨aop⟩与⟨rop⟩分别是双目算术和关系运算符，优先级前者高于后者。

d⟶[a-z]+

⟨temp⟩⟶t[0-9]+

l⟶l[0-9]+ //

k⟶[0-9]+

⟨aop⟩⟶+|-|*|/

⟨rop⟩⟶<|<=|>|>=|==|!=

KEY⟶ LABEL | GOTO | IF | THEN | ELSE | PAR | CALL | RETURN | ASG | AOP | LOAD | STORE | ADDR | NOP | define

AST 上优化：常量折叠（部分）：例如 int x = 1 + 1;，Clang 直接在 AST 层面算出 x = 2，生成的 IR 里直接就是 2。死代码消除（语法级）：例如 if (false) { ... }，Clang 在生成 IR 时根本不会为 {...} 部分生成任何指令。字符串合并：相同的字符串字面量在 AST /IR 生成时会被合并为一个全局变量。

define 上优化：SROA（Scalar Replacement of Aggregates）：这是最重要的早期优化。它试图把结构体或数组的 alloca 拆散，变成标量。简单的窥孔优化：比如删除明显的冗余指令（虽然此时因为全是内存操作，能做的优化有限）。

SSA 上优化：全局值编号（GVN）：发现 %a = x + y 和 %b = x + y 是重复计算，删除一个。常量传播：如果 %x = 10，那么 call @func(%x) 变成 call @func(10)。循环优化：循环展开、循环向量化、循环不变量外提。这些都需要 SSA 形式才能高效进行。函数内联：虽然可以在早期做，但通常结合常量传播在后期大规模进行。

寄存器分配：

LLVM IR 阶段（前端+中端）：

使用无限多的虚拟寄存器（%v0, %v1, ...）。

此时没有寄存器分配的概念，只有 SSA 值的定义和使用。

指令选择（Instruction Selection）：

IR 被转换为 机器 IR（MIR）。此时指令变成了目标架构的指令（如 ADDrr），操作数依然是虚拟寄存器。

寄存器分配（Register Allocation）：

位置：这是后端流水线中的一个 Machine Pass

`LABEL` l 为基本块入口；

`GOTO`、`IF`、`RETURN`、`CALL` 为基本块出口

基本块是单入口单出口的指令块。控制流不可能转入到基本块内部（无 `LABEL` 指令），也不可能从基本块内部转出。

控制流图（CFG）由基本块为结点，转移关系为边的有向图，其中有唯一初始结点。若要求程序是结构化块则有唯一终结结点。初始结点的入口就是程序入口，且默认标记为函数名。终结结点的出口是程序出口。

如果程序中的每个 q 都最多赋值一次，那么这样的程序是单赋值形（SSA）。临时变量是单赋值的。而源程序变量不一定，需要专门分析处理。

SSA 程序的 CFG 的需要 ϕ 指令为入口指令。

文法：不要函数做参数，不要函数声明嵌套

抽象语法树 QAST

赵银亮

28/4/26

是扁平化的 AST。通过层次体现优先级，而不是插入非终结符结点；通过

无论是编译器还是编程语言，文法是前提。先有文法才有它们。因此，将 AST 与文法绑定（扁平化的代数数据类型）是合理的。而在工程中往往考虑到多个源语言，才综合出来一种通用的 AST 节点表示形式，不是最基本的形式。

QASTNode:

$P \longrightarrow B$

NodeP:(left: NodeDList　right:NodeDlist　next: NodeSList)

NodeB:(left:NodeDList　right:NodeSList)

$T \longrightarrow$ int | void

NodeTPrim:(name:int/void　value:NUM/VOID)

NodeTArray:(left:NodeE　right:NodeTArray/NULL)

//维长表达式、数组类型或 NULL，NULL 表示是最后一维

$D \longrightarrow TV \mid HB$

$H \longrightarrow T\boldsymbol{d}(\check{A})$

$B \longrightarrow \{\check{D}\check{S}\}$

NodeDVar:(name:d.name　left:NodeT　right:NodeTArray/NULL)

//数组名、元素类型、数组类型，NULL 表示简单变量

NodeDFunc:(name:d.name　left:NodeT　right:NodeAList　next:NodeB)

//函数名、返回类型、形参表、体块

$S \longrightarrow ; \mid E; \mid \text{if}(C)S \mid \text{if}(C)S \text{ else } S \mid \text{while}(C)S \mid \text{return } E; \mid B$

NodeNOP:(name:NOP)

NodeSExpr:(left:NodeE)

NodeIF:(left:NodeC　right:NodeS　NodeS)

NodeWHILE:(left:NodeC　right:NodeS)

NodeRETURN:(left:NodeE)

$V \longrightarrow \boldsymbol{d} \mid V[E]$

$E \longrightarrow \boldsymbol{i} \mid V \mid \boldsymbol{d}(\check{R}) \mid \boldsymbol{u}E \mid E\boldsymbol{o}E \mid (E)$

NodeNUM:(value:i.value)　//[0-9]+

NodeID:(name:d.name)　//[a-zA-Z]+

NodeArrayAccess:(left:NodeArrayAccess/NodeID　right:NodeE)

NodeCall:(name:d.name　right:NodeRList)

NodeUOP:(name:u.name　left:NodeE)　//UOP={−}

NodeAOP:(name:o.name　left:NodeE　right:NodeE)　//AOP={+,−,×,/,=,+=,=+}

$\check{D} \longrightarrow \varepsilon \mid \check{D}D;$

$\acute{A} \longrightarrow \varepsilon \mid \acute{A}A;$

$\check{S} \longrightarrow S \mid \check{S};S$

$\check{R} \longrightarrow \varepsilon \mid \check{R}E,$

NodeDList:(left:NodeD right next:NodeDList)
NodeAList:(left:NodeA right: next:NodeAList)
NodeSList:(left:NodeS right next:NodeSList)
NodeRList:(left:NodeR right next:NodeRList)

$A \longrightarrow T\boldsymbol{d} \mid T\boldsymbol{d}[]$

NodeAFunc:(name:d.name left:T.type right:NodeTList)
NodeAVar:(name:d.name left: NodeT) //简单变量和数组

$C \longrightarrow E \mid E\boldsymbol{r}E \mid !C \mid C\&\&C \mid C||C \mid (C)$

NodeNZ:(name:NZ left:NodeE)
NodeROP:(name:r.name left:NodeE right:NodeE)
NodeUOP:(name:! left:NodeC)
NodeAND:(name:&& left:NodeC right:NodeC)
NodeOR:(name:|| left: :NodeC right:NodeC)

注：QAST 结点与 QL 文法及语义对应。

寄存器分配

在 ARM64（AArch64）目标机上，固定寄存器（Fixed Registers） 是指由 AAPCS64 调用约定、硬件架构或操作系统 强制指定用途、寄存器分配器绝对不能重新分配或溢出 的寄存器。

寄存器	别名	作用	分配器行为
SP (x31)	Stack Pointer	栈指针，指向栈顶	完全固定，永不参与分配
PC	Program Counter	程序计数器，指向下一条指令	硬件专用，程序不可直接写，编译器不分配
XZR (x31)	Zero Register	零寄存器，读恒为 0，写丢弃	与 SP 共用编码，硬件固定
X29	FP (Frame Pointer)	帧指针，栈帧回溯	固定（除非开启-fomit-frame-pointer）
X30	LR (Link Register)	链接寄存器，存返回地址	固定，函数调用专用

二、AAPCS64 调用约定固定寄存器

这些寄存器的用途由 ABI 强制规定，寄存器分配器必须保留、不能重新分配：

1. 参数 / 返回值寄存器（调用者保存）

X0~X7：函数参数（前 8 个）、返回值（X0）

分配器：进入函数时已固定赋值，不重新分配

X8：间接返回值地址（结构体）、系统调用号

分配器：固定用途，不分配给普通变量

2. 平台 / OS 保留寄存器

X18：平台保留寄存器（TLS 线程指针）

Linux/Android：用作 TPIDR_EL1 线程指针

绝对固定，永不分配

X16, X17（IP0, IP1）：临时 / 链接器使用

调用间隙、 veneer 跳转专用，通常不分配给用户变量

3. 被调用者保存寄存器（必须保护）

X19~X28：被调用者必须保存 / 恢复

分配器：可分配，但必须保存原值（不算 “固定”，但受严格约束）

三、浮点 / SIMD 寄存器（V 寄存器）

V0~V7：浮点 / SIMD 参数、返回值

固定传参用途，不重新分配

V8~V15：被调用者保存

V16~V31：调用者保存（可自由分配）

四、寄存器分配器的完整处理规则

绝对固定（永不分配）：

SP, PC, XZR, X18, X29(FP), X30(LR)

调用约定固定（进入时已绑定）：

X0~X8, V0~V7

分配器：不重新分配，只在函数内临时使用后恢复

可分配但受约束：

X19~X28, V8~V15

分配器：可用，但必须保存 / 恢复原值

完全可用：

X9~X15, X16~X17, V16~V31

五、总结：ARM64 固定寄存器清单

真正 “固定、永不重新分配 / 溢出” 的核心寄存器：

SP (x31), X29 (FP), X30 (LR), X18 (TLS), XZR

X0~X8（参数 / 返回）、V0~V7（浮点参数）

寄存器分配器行为：

固定寄存器全部标记为 “已占用”，分配器完全避让，

只在剩余可用寄存器（X9~X15, X19~X28, V8~V31）中分配。